Software Testing and Pytest Practical Guide for SmartCare

Software Technology 1 — Assignment 3

Learning how to design, write, run and interpret tests before Stage 7

PART I — TESTING THINKING

1. Execution is not verification

The SmartCare Starter Walkthrough explains the supplied application. This guide teaches how to investigate its behaviour through tests. Stage 7 Testing and Verification then specifies the assessed work and evidence to produce in the existing group repository.

Running `main.py` shows that the synthetic demonstration executes and prints an appointment. A plausible-looking line of output does not establish that every required rule has been checked. Similarly, a green pytest result can be produced by a test that never examines the behaviour it claims to verify.

The central question is: **What required behaviour does this test actually prove?** A useful answer identifies the scenario, observable result and supporting contract. A passing test is evidence for those conditions; it is not proof that the whole program is correct.

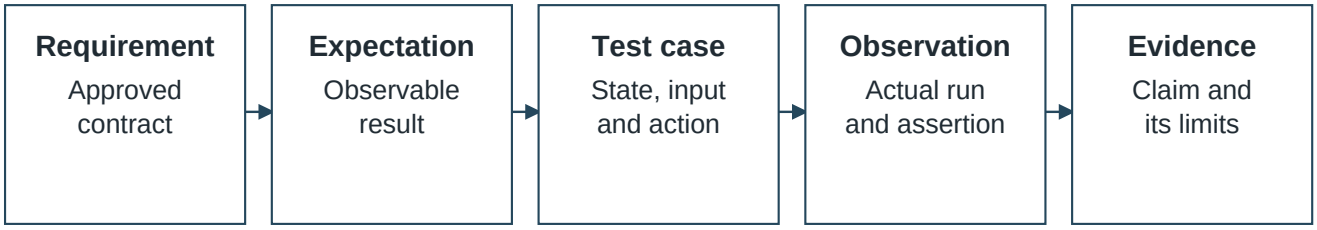


Figure 1. The expectation comes from a requirement, not from whatever the implementation happens to return.

Verification asks whether the implementation meets its specified requirement. Validation asks whether that requirement or product meets the intended need. For a practice-only duration display, verification could check that one minute is shown as sixty seconds. Validation asks whether seconds are actually the useful display for its intended reader. Correct arithmetic cannot settle that usability question.

Stage 7 mainly gathers verification evidence against the published SmartCare baseline. The group still needs to understand the purpose of the requirement rather than invent a convenient expectation.

Practice examples in this guide teach mechanics; they are not completed Stage 7 answers. They use the already supplied date parser or isolated teaching functions, not patient-registration, conflict or cancellation test suites. Keep the original activity/evidence record for the group's own assessed reasoning.

2. Reason about the case before writing code

A **test case** is a reasoned scenario: starting state, input, action and expected observation. **Test code** expresses that design so it can run repeatedly. Begin with the case; choosing a test-function name is not a substitute for deciding what should be true.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

Consider this isolated teaching contract: a conversion function receives a non-negative integer number of minutes and returns seconds. Negative minute counts are rejected with `ValueError`. No upper limit is specified. This is a practice contract, not a new SmartCare requirement.

Design question	Practice reasoning
What is the contract?	Convert permitted minute counts to seconds; reject negative counts.
What starting state is needed?	None beyond a fixed input; this function has no stored state.
What input and action?	Supply one minute and call the conversion function once.
What should be observable?	A returned numeric value of sixty.
What assertion would check it?	Equality with the independently calculated expected value.
What would the case miss?	Other inputs and rejection behaviour; one case does not cover the whole contract.

The reasoning route is contract → starting state → input → action → expected observation → assertion. Derive the expectation independently. Copying the actual output into the expected value can make a faulty implementation approve itself.

A positive case exercises permitted behaviour. A negative case exercises a rejected input, action or state. A boundary case examines the edge where a real rule changes. These categories can overlap: a just-outside value may be both negative and a boundary case.

For the conversion contract, zero is the included lower boundary and a negative integer lies outside it. A very large value is not automatically a boundary: no maximum was specified.

For a separate practice-only display-label rule, suppose trimmed text must contain a character. `"Ready"` is valid, `""` is invalid, and `"  "` examines the distinction between characters and meaningful non-blank content. This is a conceptual label example, not a completed test for a SmartCare entity.

“Unusual” and “boundary” are different descriptions. Use the published rule to identify the edge; do not add arbitrary maximum lengths, clinical rules or past-date restrictions merely to create more tests.

3. Observe behaviour and choose an assertion

Tests can observe a return value, an object reference, public state, list contents, a documented exception or state that should remain unchanged. Select the observation that matches the claim. Prefer public behaviour over direct inspection of private dictionaries when the public interface already exposes the needed evidence.

Assertion pattern	What it establishes	What it does not establish
`assert True`	Execution reached this statement.	No property of the result is checked.
`assert result is not None`	A result exists.	It may be the wrong value, object or state.
`assert result == expected_value`	Equality for the compared values.	Equality is not necessarily object identity; unrelated fields may remain unchecked.
`assert result is expected_object`	Both references identify the same object.	It does not check that object's fields or the correctness of an entire workflow.

There is no universally strongest assertion. Identity is suitable when the contract requires the same object; value equality is suitable when the contract requires a computed value. Several focused assertions can describe one coherent result, provided each has a reason.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

This small Python identity example uses ordinary lists, not SmartCare registration:

```
original = [2, 4]
alias = original
copy = list(original)

assert alias is original
assert copy == original
assert copy is not original
```

The copy has equal contents but is a different container. SmartCare also distinguishes a new list container from the domain objects referenced inside it. Its registered-object relationships make identity a useful observation, but an identity assertion alone does not prove all required workflow behaviour.

Ask whether a relevant incorrect result could still satisfy the assertion. A positive-length list may contain the wrong records. A result of the right type may contain the wrong value. An exact value assertion may be irrelevant if the approved contract only promises membership rather than ordering.

Keep the oracle—the expected result—linked to the contract. Assert the promised behaviour, not incidental choices such as a private dictionary's variable name or an undocumented implementation detail.

4. Domain evidence and collaboration evidence

A domain test concentrates on an object's own behaviour or invariant. In SmartCare, `Patient`, `Practitioner` and `Appointment` live in `domain.py`; `AppointmentStatus` describes the supported states. Domain observations concern their public fields, validation and operations.

A service test examines collaboration. The service may work with `SmartCareDataStore` and several domain objects before producing an observable workflow result. This is broader evidence than successfully constructing one object.

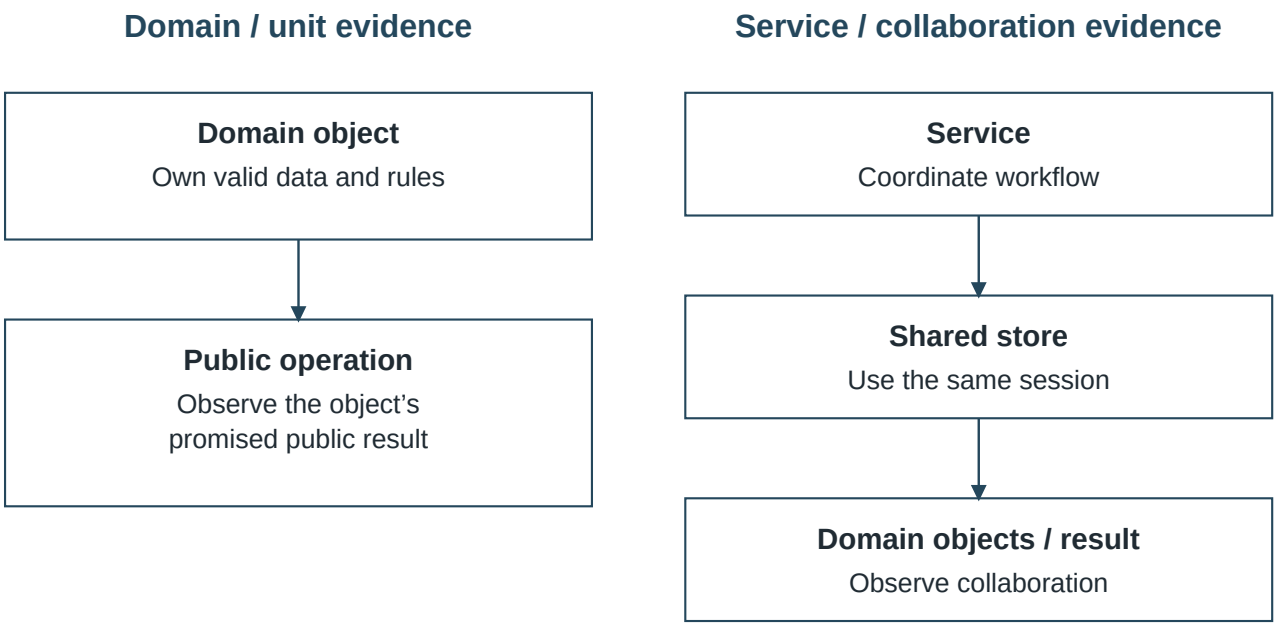


Figure 2. Choose the scope of the claim before choosing the test setup.

A constructor-only check can show that a domain object accepts an input or protects a local invariant. It does not by itself establish that a service has registered that object, retained it in the shared session or coordinated the required collaboration. Do not label partial evidence as proof of an entire workflow.

Conversely, a workflow test may reveal a problem across several components without immediately locating its cause. Smaller domain checks can help distinguish local rules from coordination problems. The two levels complement one another; they are not competing methods.

The current architecture gives each service a reference to the same store within a workflow. A service test should reason about that shared starting state. Separate stores would represent separate sessions, even if their synthetic IDs happened to match.

Use real, simple domain objects and the supplied store for ordinary SmartCare collaboration tests. Advanced mocking frameworks or replacement repository layers are not needed for this unit. The exact assessed cases remain the group's design work under the Stage 7 handout.

5. Arrange–Act–Assert makes the design visible

Arrange establishes only the state and data needed for the case. **Act** performs the target behaviour. **Assert** checks its expected observation. This structure helps a reviewer separate preparation from the rule being tested.

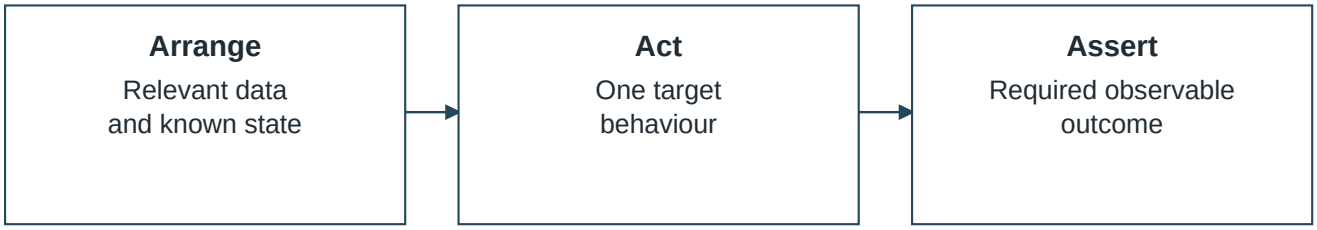


Figure 3. Setup, target action and expected observation have distinct roles.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

Use this isolated teaching function; do not add it to `smartcare/`. Its practice contract expects integer minute counts and rejects negative counts. Type-validation extensions are outside this exercise.

```
def minutes_to_seconds(minutes):
    if minutes < 0:
        raise ValueError("Minutes cannot be negative")
    return minutes * 60
```

The annotated test exposes the three stages:

```
def test_one_minute_in_seconds():
    minutes = 1                  # Arrange
    expected = 60
    result = minutes_to_seconds(minutes)  # Act
    assert result == expected    # Assert
```

The same test in normal final style needs no repeated explanatory comments:

```
def test_one_minute_in_seconds():
    result = minutes_to_seconds(1)
    assert result == 60
```

Use either version, not two functions with the same name in one file. In pytest, repeated names can replace an earlier definition rather than create two useful cases.

One target behaviour does not mean one line of setup or exactly one assertion. A state-changing operation may require a few related observations. Avoid combining unrelated behaviours into one long test whose first failure hides the rest.

The expected sixty is established from the practice contract. Calling the same conversion function again to calculate the expected result would remove that independence.

6. Rejection is behaviour, too

A permitted result is only part of many contracts. Correct behaviour may instead be to reject an input or action with a documented exception. `pytest.raises` expresses that expectation. If no exception is raised, the test fails; an unexpected exception is not silently converted into success.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

With the practice conversion function from section 5, this syntax checks its negative-input contract:

```
import pytest

def test_negative_minutes_rejected():
    with pytest.raises(ValueError):
        minutes_to_seconds(-1)
```

Place only the operation expected to raise inside the block. Otherwise an unrelated setup error of the same type could satisfy the test. Avoid `pytest.raises(Exception)` when a specific error is promised; it may accept an unintended exception, including an error in the test itself. A broad `try/except` that simply ignores every error is weaker still.

An exception does not automatically establish that stored state is intact. A function might change data and then raise. Check both promises where the contract requires rejection without mutation.

This independent list exercise illustrates that second observation without a SmartCare workflow:

```
def append_number(numbers, value):
    if value < 0:
        raise ValueError("Negative values are not accepted")
    numbers.append(value)

def test_rejected_append_preserves_list():
    numbers = [2, 4]
    before = list(numbers)
    with pytest.raises(ValueError):
        append_number(numbers, -1)
    assert numbers == before
```

The post-block assertion is reached after the expected exception. It checks contents against a separate snapshot rather than another reference to the same mutable list. This teaches a checking technique; it is not a supplied cancellation, registration or conflict test.

In SmartCare, choose exceptions from the current public contract. A service boundary, a store lookup and an intentionally incomplete TODO can have different outcomes; do not assume one exception fits every operation.

7. Every test establishes its own state

An independent test can run first, last, alone or with the suite. It does not need another test to create records or clean up previous changes. Shared mutable setup can make apparently correct cases depend on collection order.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

The following module is deliberately unreliable:

```
numbers = []

def test_add_number():
    numbers.append(2)
    assert numbers == [2]

def test_new_list_is_empty():
    assert numbers == []
```

Run together in this order, the second test sees the first test’s data and fails. Run the second alone in a fresh process and it passes. Reversing the selected tests can also change the outcome. Those differences expose a shared-state problem, not a stable verification result.

Fresh local setup removes the dependency:

```
def test_add_number():
    numbers = []
    numbers.append(2)
    assert numbers == [2]

def test_new_list_is_empty():
    numbers = []
    assert numbers == []
```

Both tests now establish the state they need. An ordinary helper that returns a fresh list is also acceptable. Fixtures, introduced later, are an optional way to express repeated setup; they do not remove the need to understand freshness.

For a SmartCare service case, use a fresh store for each independent test, but share that one store among the services within the case. Freshness across tests and sharing within a workflow serve different purposes.

Changing execution order is a useful check of independence. It does not replace inspecting the setup: two tests may pass in several orders while still sharing data unnecessarily.

8. Repeatable data and regression evidence

Use fixed synthetic IDs, names and datetimes when a case needs them. A fixed `datetime(2026, 10, 1, 9, 0)` is easier to reason about than a value obtained from the current clock. It also avoids a test that changes with the date, local timezone or machine speed. Do not use real patient or personal information.

Keep tests free of unnecessary network calls, external services and machine-specific files. Repeatable inputs make a failure easier to reproduce and compare. If a dependency is genuinely required, its effect must be understood rather than hidden inside the setup.

Regression testing checks that earlier required behaviour still works after a change. When a relevant defect is found, retain a meaningful case that exposes it, verify the repair and keep that case in the suite for later changes.

```
Defect observed
    ↓
Test detects the relevant incorrect result
    ↓
Implementation corrected and test passes
    ↓
Test retained in the suite
    ↓
Later change → rerun earlier contracts
```

Running pytest twice without preserving any useful earlier checks is not the whole idea of regression testing. The value lies in what the retained assertions protect.

Judge a suite by meaningful coverage of permitted and rejected behaviour, boundaries, state transitions, identity, errors, collaboration and state preservation. Raw case count is a poor substitute. Near-duplicates may repeat the same observation while missing an important different scenario.

Nor does a longer test automatically provide better evidence. Several unrelated operations in one case can make a failure harder to interpret. Give every case a distinct claim and explain its added value beyond existing scaffold tests.

Stage 7 continues the accumulated suite from Stage 6. Do not remove an earlier assertion solely to obtain green output, invent a business rule to increase the count or replace public observations with unnecessary private-state checks. Record the actual result, including unresolved failures.

PART II — PYTEST IN PRACTICE

9. Files, discovery and the baseline

Python application and pytest commands normally run from `project/`; normal Git commands run from the repository root above it. The actual supplied application tree is:

```
project/
├── README.md
├── requirements-dev.txt
├── main.py
├── smartcare/
│   ├── __init__.py
│   ├── domain.py
│   ├── data_store.py
│   ├── services.py
│   ├── queries.py
│   └── console.py
└── tests/
    └── test_starter.py
```

Only `test_starter.py` is supplied here. In the assessed project, Stage 6 creates service tests; Stage 7 creates or extends `tests/test_domain.py` and extends `tests/test_services.py`. These are accumulated test files, not new applications.

pytest discovers matching test files and functions, collects the runnable cases, then executes them. Use names such as `test_practice_parser.py` and `test_parse_datetime`. The usual convention in this unit is a `test_` filename and a `test_` function name. Tests belong in test files; do not place them in `main.py` or mix them with stage-record responses.

Use the teaching team's confirmed interpreter and environment. The examples were verified with Python 3.13 and the dependency file's pinned pytest 8.3.5. From the repository root:

```
cd project
python --version
python -m pip install -r requirements-dev.txt
python -m pytest
```

The unchanged supplied starter collects eight cases and reports `8 passed`. That baseline confirms scaffold behaviour only. It deliberately does not call the seven incomplete operations.

The assessed Stage 7 baseline is different: use the reviewed Stage 6 application with its completed service work and previous tests. Record its actual count; this guide does not prescribe an assessed count. If that baseline fails, preserve the observation and resolve it before building new evidence.

10. A first complete helper test

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

Use a disposable learning copy of the starter for practice. It is separate from the assessed group repository; do not replace that repository's `project/`. In the learning copy's `project/`, create a temporary `practice/` directory:

```
python -c "from pathlib import Path; Path('practice').mkdir(exist_ok=True)"
```

Create `practice/test_practice_parser.py` in the editor with this complete example:

```
from datetime import datetime

from smartcare.console import parse_date_time


def test_parse_datetime():
    text = "2026-10-01 09:00"
    expected = datetime(2026, 10, 1, 9, 0)
    result = parse_date_time(text)
    assert result == expected
```

The imports make the expected datetime type and already supplied helper available. The function name makes this a discoverable test. Its fixed text and expected value are Arrange, the parser call is Act, and equality is Assert. No patient, practitioner, booking or cancellation workflow is exercised.

Run only that function from `project/`:

```
python -m pytest practice/test_practice_parser.py::test_parse_datetime
```

The verified result is one passing test. This shows that this helper converts the chosen correctly formatted text to the expected datetime. It does not prove that every malformed input is rejected or that SmartCare booking is correct.

`python -m pytest -v` gives each collected node's name and outcome. A node ID combines a file path with a function name separated by `::`; it is not a Python import path.

Keep practice files separate and temporary. Their role is learning pytest, not supplying the group's assessed domain/service evidence. The mini-lab explains how to remove its practice file and confirm the scaffold baseline afterwards.

11. Practice with pytest.raises

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

The supplied parser accepts exactly `YYYY-MM-DD HH:MM`. A `T` separator is outside that helper contract. Add `import pytest` at the top of the practice parser file and append this second test:

```
import pytest


def test_parser_rejects_wrong_separator():
    with pytest.raises(ValueError):
        parse_date_time("2026-10-01T09:00")
```

Keep the earlier imports and positive test. This excerpt is an addition, not a replacement file. Running the practice parser file now produces two passing cases:

```
python -m pytest practice/test_practice_parser.py
```

The `with` block passes when the helper raises the expected type. It accepts a subclass of that type as well. It is not checking every possible invalid string or promising a particular error message unless the contract requires one.

Two intentionally mistaken variants show why the expectation matters. Expecting `ValueError` for the valid text `"2026-10-01 09:00"` fails because no exception should occur. Expecting `LookupError` for `"not a date"` fails because this parser raises `ValueError` instead.

Abbreviated lines from the verified practice runs were:

```
Failed: DID NOT RAISE <class 'ValueError'>
ValueError: Use date/time format YYYY-MM-DD HH:MM
```

They come from separate runs, not one combined traceback. The first is a mistaken rejection expectation; the second is an unexpected type relative to a mistaken test. Neither authorises changing the correctly supplied parser.

Keep expected-error operations narrow. If unrelated setup is placed inside `pytest.raises`, it may raise the expected type for the wrong reason. Check any promised state preservation after the block, as in the independent list exercise, rather than assuming rejection proves it.

12. Select tests and read pytest's outcomes

Use a focused run while developing a case, the related file after connected work, and the full available suite before commit, review and integration. Selection is a development convenience; a passing selected subset is not evidence that omitted tests passed.

Command from project/ (one per row)	Purpose
`python -m pytest`	Collect and run the full available suite.
`python -m pytest -v`	Full suite with named individual outcomes.
`python -m pytest tests/test_starter.py`	Run the supplied scaffold file.
`python -m pytest practice/test_practice_parser.py`	Run the temporary helper file while it exists.
`python -m pytest practice/test_practice_parser.py::test_parse_datetime`	Run one specific practice function.
`python -m pytest -k "parse"`	Select matching names; inspect what was selected and deselected.

Once the group's own files exist, the same file-selection syntax applies to:

```
python -m pytest tests/test_domain.py
python -m pytest tests/test_services.py
```

Those files are not present in the untouched starter. A missing-file message means the requested path could not be run; it is not a failing domain or service observation.

Outcome	Read it as
`PASSED`	The collected case completed without an unmet test expectation. Its evidence is limited to its setup and assertions.
`FAILED`	A running test encountered an unmet assertion, missing expected exception or unexpected exception. Inspect the cause.
`ERROR`	Collection or setup could not complete, for example an import failure or unavailable fixture. The intended observation may never have been made.

These labels do not identify product versus test defects automatically. A `NameError` inside a test may appear as FAILED although the test itself forgot an import. An import error during collection appears as ERROR and prevents the intended cases from executing.

"No tests ran" and "all tests passed" are different reports. Check the collected count, selected names, failures, errors and deselections—not merely the colour of the final line.

13. Read a failure before choosing a repair

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

For a deliberately wrong practice assertion, parse `"2026-10-01 09:00"` and assert that the returned day is 2. The helper correctly returns day 1. An abbreviated actual failure is:

```
FAILED practice/test_practice_failure.py::test_parse_day
E   assert 1 == 2
```

The named test identifies the case. Its failing assertion compares observed 1 with expected 2. Read the traceback to locate the test line and any application call involved, then compare the expectation with the helper contract and input. Here the test is wrong; the parser does not need a repair.

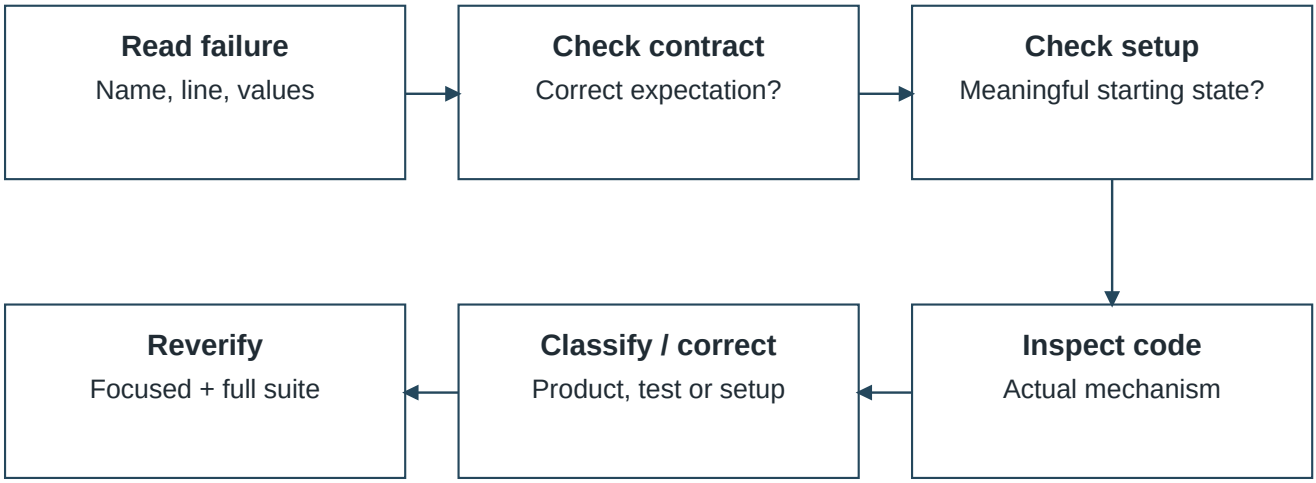


Figure 4. A traceback locates the failure path; it does not establish the root cause on its own.

Cause	Safe practice example	Correct response
Product defect	The section 5 teaching conversion is changed to multiply by 30 although its contract specifies 60.	Restore the correct practice implementation and rerun its test.
Test defect	A parser test expects day 2 from text that explicitly names day 1.	Correct the mistaken expectation; do not alter production behaviour.
Setup/environment defect	A test module imports a nonexistent practice module and cannot be collected.	Correct the import/environment, then actually run the intended cases.

Preserve the failing version and observation. Check the approved contract, test setup and relevant implementation before deciding which artefact to change. If the cause is unresolved, record that uncertainty rather than declaring the run successful.

After a justified correction, rerun the focused case and the full regression suite. Changing production code simply to satisfy an invented expectation replaces engineering judgement with test accommodation.

14. Optional fixtures: fresh repeated setup

Local setup is sufficient for Assignment 3. A fixture is an optional pytest mechanism for providing repeated setup to tests that request it. pytest calls the fixture and supplies its returned value through the test parameter with the matching name.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

This complete independent list example belongs in a temporary practice file, not `smartcare/`:

```
import pytest


@pytest.fixture
def numbers():
    return []


def test_fixture_can_start_a_list(numbers):
    numbers.append(2)
    assert numbers == [2]


def test_fixture_starts_empty(numbers):
    assert numbers == []
```

The default fixture scope is one test function. Each test receives a freshly created list here, so the first test's append does not affect the second test. Both pass together, separately and in reversed selection order.

The decorator `@pytest.fixture` marks `numbers` as a setup provider. The parameter in each test requests that provider; it is not an ordinary caller passing an argument manually. A misspelt parameter/fixture name can produce a setup error before the test body runs.

Freshness depends on what the fixture creates. Returning the same mutable global list would still share state, even with a function-scoped fixture. Inspect the body rather than assuming the decorator guarantees independence.

An ordinary helper returning a new list, or explicit `numbers = []` inside each test, is equally acceptable. Prefer whichever makes the case easiest to read. No fixture count or fixture use is required for Stage 7.

Keep this at the basic level: no session-scoped mutable fixtures, autouse setup, fixture factories or advanced dependency chains are needed. Fixtures organise setup; they do not decide the requirement or assertion.

15. Optional parametrisation and coverage limits

Parametrisation expresses one behaviour over several meaningful input/expected-result pairs. pytest collects a separate case for each pair. It is useful when the intention stays the same and the data varies; unrelated behaviours may be clearer as separate named tests.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

This complete parser practice module checks three valid strings:

```
from datetime import datetime

import pytest

from smartcare.console import parse_date_time


@pytest.mark.parametrize("text, expected", [
    ("2026-10-01 09:00", datetime(2026, 10, 1, 9, 0)),
    ("2028-02-29 08:15", datetime(2028, 2, 29, 8, 15)),
    ("2026-12-31 23:59", datetime(2026, 12, 31, 23, 59)),
])
def test_parser_valid_examples(text, expected):
    assert parse_date_time(text) == expected
```

The parameter names match the function arguments; each tuple supplies one pair. This verified example produces three passes. It is not three different behavioural claims, nor a required Stage 7 parametrisation pattern. Both fixtures and parametrisation are optional.

Coverage indicates which code executed during a test run. It can help identify an unexecuted branch, but does not establish that the requirement, expected result, assertion or boundary selection was correct.

```
100% executed lines ≠ 100% correct software
```

A call followed by an irrelevant assertion can execute code without checking its promised result. Several input combinations may reach the same lines while exercising different behaviour. Coverage cannot decide whether the approved product meets the intended need either.

No coverage tool installation, plugin or percentage target is required here. If used as supplementary evidence, explain what the measure omits. A concise statement of behavioural limits is more useful than an unexplained percentage.

PART III — TESTING SMARTCARE

16. Prepare a case at the right boundary

Return to the actual application before designing assessed tests. Read `domain.py`, `data_store.py` and the completed Stage 6 `services.py`, then identify which public contract the case will observe. Keep query/menu TODOs outside Stage 7 work.

For a service-level case, the preparation is conceptual rather than a supplied implementation:

```
Create a fresh SmartCareDataStore
    ↓
Create services sharing that one store
    ↓
Establish only the required synthetic objects and state
    ↓
Perform one target behaviour
    ↓
Observe its documented result and any protected state
```

All services in that workflow must work with the same session. Otherwise an object prepared through one service may not exist in the store seen by another. Start a new store for the next independent case; do not depend on a previous test's registrations.

Domain tests examine constructor validation, initial public state, public operations and invalid state transitions. Use the actual class contracts rather than inventing fields or rules. `Patient` and `Practitioner` expose their IDs through properties; `Appointment` exposes participant references, time and status. Type hints alone do not enforce validation; inspect the supplied runtime checks and docstrings.

Service tests add evidence about the shared store, actual registered objects, coordination, service errors, retained state and the workflow result. A constructor observation and a completed service observation are different claims. Explain which one the case provides.

Ask these questions while planning, without treating them as completed test designs:

Scope	Planning question
Domain	Which local rule or public state is being observed?
Service	Which collaboration must occur to meet the workflow contract?
Rejection	Which documented exception applies at this boundary, and what must remain unchanged?
Identity	Does the contract require the registered object or only an equal value?
Added evidence	What does this case establish beyond the supplied checks?

`ValueError`, `LookupError` and a store's `None` result are not interchangeable. An intentionally incomplete operation raises `NotImplementedError`; it is not a correctly implemented rejection path. Stage 7 depends on verified Stage 6 work, not merely the original eight scaffold passes.

17. Trace the claim, implementation and observation

Traceability connects the approved requirement to its verification evidence. A TEST ID identifies a case; it does not create a new business requirement. Keep canonical requirement/acceptance IDs with their published meanings. If a supporting public contract has no canonical acceptance ID, quote it rather than reusing an unrelated ID.

```
FR-XX → AC-XX.X → TEST-XX
    ↓
file and test function → expected behaviour → actual result
```

The labels above show the structure only. They are not entries to paste into the assessed Stage 7 matrix.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

For the parser teaching case, the contract is the supplied helper's strict format/conversion behaviour, not an assigned FR/AC. A practice evidence note could read:

Evidence item	Practice note
Contract	Supplied `parse_date_time` docstring; no invented assessed AC number.
Practice identifier	`PRACTICE-TIME-01`, a learning label only.
Case and location	Valid fixed text; `practice/test_practice_parser.py::test_parse_datetime`.
Expected result	The corresponding datetime, derived independently from the text.
Observed result	One passing focused case in the practice run.
Limit	No registration, conflict, cancellation or retained-history requirement verified.

For assessed work, follow this reasoning worksheet independently:

1. Read the requirement and its exact public contract.
2. Identify one behaviour the case will claim.
3. Establish the required starting state.
4. Choose a positive, negative or boundary purpose.
5. Identify the observable result.
6. Select the strongest useful assertion for that claim.
7. Identify the expected exception, if any.
8. Decide whether existing state must be checked after rejection.
9. Assign a TEST ID before generating or writing code.
10. Write the pytest implementation.
11. Run the focused case.
12. Run the full accumulated suite.
13. Record the actual version, command, directory and result.

Distinguish supplied from newly authored tests. Link actual functions, reviewed versions and observations; a name or screenshot alone does not prove a criterion.

18. Review suggestions and troubleshoot the right cause

GitHub Copilot can suggest scenarios, edge cases, assertions or code. The group must decide whether those suggestions match the published requirement. Write the human test intention first; an AI-generated expected result is not automatically an independent oracle.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

The following is an illustrative candidate, not a claimed Copilot transcript. Assume it is added to a practice file with the parser import:

```
def test_candidate_parser_check():
    result = parse_date_time("2026-10-01 09:00")
    assert result.year == 2026
```

It passes for the supplied parser, but only checks the year. A wrong day or time could still satisfy it. If the practice claim is complete conversion of the chosen text, the equality assertion in section 10 provides more relevant evidence. That observation does not complete the Stage 7 required critique activity.

For each real candidate, check its requirement trace, observable claim and assertion. Check whether it invents a business rule, expects the wrong exception, unnecessarily depends on private state, duplicates another case or introduces a dependency without need. Decide Accept, Modify or Reject with reasons and independently run any retained test. Do not invent an AI interaction or claim an unrun proposal passed.

Common mistake	Check or corrective action
Test not discovered	Check file/function naming and the selected path; confirm collection.
`ModuleNotFoundError` for smartcare	Check project/ working directory, selected interpreter and imports before editing application code.
`NameError` for pytest	The test may have omitted `import pytest`; inspect the failing line.
Irrelevant assertion	State the promised observation and ask what wrong result could still pass.
Order-dependent result	Look for shared mutable state; establish fresh setup.
Wrong/generic exception	Use the specific boundary’s documented type.
Invented rule	Recheck the requirement; correct the test’s oracle rather than accommodate it.
Private storage edits or duplicate cases	Prefer public setup/observations and explain each case’s added value.
Real clinical/personal data	Replace it with fixed synthetic learning data; do not share it with AI.

The observed symptom directs an investigation, not an immediate production edit. After correcting the justified cause, preserve review evidence and rerun focused plus full tests.

19. Practical mini-lab

PRACTICE ONLY — THIS IS NOT A STAGE 7 SUBMISSION TASK.

Use a fresh disposable starter copy and the approved environment. These practice counts describe that learning copy, not the Stage 6 group baseline. Do not replace the assessed project or copy practice answers into it.

1. Enter its `project/` and run the supplied suite:

```
cd project
python -m pytest
```

2. Open `tests/test_starter.py` and locate `test_registration_and_booking`.
3. Identify its Arrange, Act and Assert from the actual source. Explain the distinction without copying it as newly authored assessment evidence.
4. Create the temporary practice directory if needed:

```
python -c "from pathlib import Path; Path('practice').mkdir(exist_ok=True)"
```

5. Create `practice/test_practice_parser.py` with the positive helper test and imports in section 10.
6. Run that one test; the verified result is one pass:

```
python -m pytest practice/test_practice_parser.py::test_parse_datetime
```

7. Deliberately change the expected datetime's day from 1 to 2, leaving the input text unchanged.
8. Rerun the same focused command. It now fails: read the test name, failing line, actual datetime and mistaken expected datetime. Do not change `console.py`.
9. Restore day 1 and rerun the focused command to obtain one pass again.
10. Add `import pytest` and the safe wrong-separator rejection case from section 11.
11. Run the practice file; both cases pass:

```
python -m pytest practice/test_practice_parser.py
```

12. Run the full available suite. With only these two practice cases added, the verified learning-copy result is ten passes: eight scaffold cases plus two helper cases.

```
python -m pytest
```

13. Explain in one sentence what each helper case actually checks and what it does not establish about SmartCare workflows.

Remove only the temporary parser practice file after the exercise, then confirm the learning-copy scaffold baseline again:

```
python -c "import os; os.remove('practice/test_practice_parser.py')"
python -m pytest
```

Post-cleanup: eight passes. Do not delete assessed tests to match this figure. Other practice files change the count. This failure/restoration exercise is unassessed practice, separate from the optional Stage 7 tutor demonstration.

20. Reference and bridge into Stage 7

The Starter Walkthrough explains the application; this guide teaches testing. **Stage 7 specifies the assessed work.** Read its handout before designing the group's cases and use the original `stage07/stage07_activities_and_evidence.md`.

```
Testing concepts + pytest practice
    → Stage 7 handout: requirements and assessed work
    → Own cases, tests and observations → original stage07 record
```

Practical reminder	Use
Naming	`test_*.py` files and `test_` functions; descriptive behaviour names.
AAA	Known starting state → one behaviour → promised observation.
Case purpose	Positive succeeds; negative rejects; boundary comes from an actual rule edge.
Observation	Value, identity, public state, contents, specific exception or protected state.
Failure reading	Name → line → actual/expected → call path → contract → setup.
Cause	Product violates contract; test has wrong oracle/setup; environment blocks execution.
Trace	Requirement/contract → TEST ID → file/function → expected/actual result and version.

Core commands from `project/`:

```
python -m pytest
python -m pytest -v
python -m pytest tests/test_starter.py
python -m pytest practice/test_practice_parser.py::test_parse_datetime
```

The focused command requires the practice file. For assessed work, select the group's actual function; run the full accumulated suite before integration.

PRACTICE EXAMPLE — NOT A STAGE 7 ASSESSMENT ANSWER

Expected-exception reminder for the supplied helper:

```
with pytest.raises(ValueError):
    parse_date_time("not a date")
```

Import pytest and the helper first. Check protected state after rejection where promised. Fixtures, parametrisation and coverage tooling are optional.

Use the current Overview, Git/GitHub Group Guide and Stage 7 handout for requirements and workflow; practice is not assessed evidence.

Further reference: official pytest 8.3 commands, assertions, fixtures and parametrisation. Use the pinned unit dependency.